

Reviewer Pack — Minimal p-Adic Valuation Rank and DPLL Proof Tree Path Length...

Entry 92f245340351 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 00:44:38 UTC

Minimal p-Adic Valuation Rank and DPLL Proof Tree Path Length Correlation

Entry ID: 92f245340351 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 00:44:38 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every CNF φ with n variables, the minimal p-adic valuation rank ($\text{valrank}(\varphi)$) of its clauses is linearly correlated with its DPLL proof tree path length ($\text{dpl}(\varphi)$), such that $\text{valrank}(\varphi) = \Theta(\text{dpl}(\varphi))$.

Rationale (proposer's reasoning):

p-adic analysis provides a different perspective on the structure of formulas. The valuation rank measures the complexity of the p-adic representation, which might reflect the depth of DPLL search trees due to the nature of the proof process. This bridge may reveal connections between the arithmetic properties of formulas and their proof complexity.

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e526b75916fccb0d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient (r) between p-adic valuation rank (valrank) and DPLL proof tree path length (dpl) for all CNF instances exceeds 0.8, with a p-value ≤ 0.05 , based on at least 30 random CNFs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=108.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2**n): # Generate 2^n clauses
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            cnf.append(clause)
        return cnf

    def p_adic_valuation_rank(cnf):
        valuation_rank = 0
        for clause in cnf:
            prime_ideals = set()
            for literal in clause:
                if literal > 0:
                    prime_ideals.add(literal)
                else:
                    prime_ideals.add(-literal)
            valuation_rank = max(valuation_rank, len(prime_ideals))
        return valuation_rank

    def dpll(cnf):
        literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]

        def solve(model, literals):
            if not literals:
                return [model]
            literal = literals[0]
            pos_model = model.copy()
            neg_model = model.copy()
            pos_model[literal] = True
            neg_model[literal] = False
```

```

        pos_solutions = solve(pos_model, literals[1:])
        neg_solutions = solve(neg_model, literals[1:])
        return pos_solutions + neg_solutions

    solutions = solve({}, literals)
    return len(solutions) > 0

n_values = [5, 10, 15, 20, 30, 40]
valranks = []
dpls = []

for n in n_values:
    cnf = generate_cnf(n)
    valrank = p_adic_valuation_rank(cnf)
    dpl = dpll(cnf)
    valranks.append(valrank)
    dpls.append(dpl)

mean_valrank = sum(valranks) / len(valranks)
mean_dpl = sum(dpls) / len(dpls)

correlation_coefficient = 0
if mean_valrank != 0 and mean_dpl != 0:
    numerator = sum((valranks[i] - mean_valrank) * (dpls[i] - mean_dpl) for i in range(len(valranks)))
    denominator = math.sqrt(sum((valranks[i] - mean_valrank)**2 for i in range(len(valranks))))
    correlation_coefficient = numerator / denominator

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(valranks),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": "" if correlation_coefficient >= 0.8 else "Pearson correlation coefficient < 0.8"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31, 100))
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Pearson correlation coefficient < 0.8' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition was not met and the critic challenged the result. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13372
2	propose	ollama_remote	glm4:latest	0	0	13016
3	preregistration	ollama_remote	glm4:latest	0	0	9515
4	novelty	ollama_remote	glm4:latest	0	0	8906
5	novelty	ollama_remote	glm4:latest	0	0	8619
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24552
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13909
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14698
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11388
10	judge	ollama_remote	glm4:latest	0	0	64833

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182807 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/92f245340351.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/92f245340351.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/92f245340351.tar.gz (if generated)